INSTITUT DE TECHNOLOGIE
Enseignement pour adultes de la Ville de Liège

TECHNIQUES DE PROGRAMMATION PROCÉDURALE

Les pointeurs en C

30 exercices progressifs · théorie, conseils et mises en pratique

NIVEAU 1 · FACILE

Mémoire et pointeurs

Adresses, pointeurs, passage par adresse

Exercices 01 – 10

NIVEAU 2 · MODÉRÉ

Structures et organisation des données

struct, tableaux de structures

Exercices 11 – 20

NIVEAU 3 · EXPERT

Allocation dynamique

malloc, realloc, free

Exercices 21 – 30

VERSION ÉTUDIANT

COURS
Programmation procédurale

PROFESSEUR
Romain Chariot

ANNÉE ACADÉMIQUE
2026 – 2027

ÉTABLISSEMENT
Institut de Technologie
Ville de Liège

Quai du Condroz, 15 – 4020 Liège
04 / 258 41 20
ps.technologie@ecl.be

AVANT DE COMMENCER

Comment utiliser ce cahier

Ce cahier t'accompagne de ton **premier pointeur** jusqu'aux **structures de données dynamiques**. Il suit les trois chapitres du cours consacrés à la mémoire. Chaque niveau s'ouvre sur une fiche théorique qui contient tout ce qu'il faut pour réaliser ses dix exercices.

Trois niveaux, trente exercices

Niveau 1 · Facile
Exercices 01 à 10 — mémoire, pointeurs, passage par adresse.

Niveau 2 · Modéré
Exercices 11 à 20 — tableaux, structures, tableaux de structures.

Niveau 3 · Expert
Exercices 21 à 30 — malloc, realloc, free et structures dynamiques.

Une méthode en quatre étapes

1 — Lire
Repère les prototypes imposés et le résultat attendu avant d'écrire une ligne.

2 — Dessiner
Schématise la mémoire : des boîtes pour les variables, des flèches pour les pointeurs.

3 — Coder
Fonctions séparées du `main`, compilation fréquente, avertissements activés.

4 — Tester
Cas normal, cas limites, entrées invalides… et détection des fuites.

Les encadrés

Rappel
Une notion vue en théorie, à garder en tête pour réussir l'exercice.

Conseil
Une piste méthodologique pour démarrer ou débloquer la situation.

Attention
Une erreur fréquente, souvent à l'origine d'un plantage.

À retenir
L'essentiel d'une fiche théorique, à relire avant une évaluation.

Question de réflexion
Une question à laquelle tu dois savoir répondre oralement.

Pour aller plus loin
Un prolongement facultatif pour ceux qui ont terminé.

Consignes générales

- Réalise chaque exercice dans un programme testable ; les fonctions demandées sont **séparées du** `main`.
- Compile toujours avec les avertissements : `gcc -std=c11 -Wall -Wextra -Wpedantic`. Un avertissement est une erreur en puissance.
- Le temps indiqué est une **estimation** pour un premier essai, explications comprises.
- Sois capable d'expliquer **chaque** `&`, chaque `*` et chaque `->` de ton code.

TABLE DES MATIÈRES

Sommaire

Comment utiliser ce cahier 2

1 NIVEAU 1 · FACILE
Mémoire et pointeurs 4

1 Fiche théorique — Mémoire et pointeurs 4

01 Observer des adresses 9
02 Déclarer, initialiser, déréférencer 10
03 Prédire avant d'exécuter 10
04 Comparer des pointeurs ou des valeurs ? 11
05 Valeur ou adresse : la preuve par l'exemple 12
06 Élever au carré en toute sécurité 12
07 Échanger deux valeurs 13
08 Ordonner deux valeurs 14
09 Convertir une durée 14
10 Rotation de trois valeurs 15

2 NIVEAU 2 · MODÉRÉ
Structures et organisation des données 17

2 Fiche théorique — Structures et organisation des données 17

11 Parcourir un tableau avec un pointeur 22
12 Inverser un tableau avec deux pointeurs 22
13 Rechercher et renvoyer une adresse 23
14 Première structure : des points dans le plan 24
15 Fiche étudiant 25
16 Catalogue : un tableau de structures 26
17 Trouver un produit par son nom 26
18 Statistiques sur le catalogue 27
19 Trier le catalogue par prix 28
20 Mini-projet : la bibliothèque 29

3 NIVEAU 3 · EXPERT
Allocation dynamique 31

3 Fiche théorique — Allocation dynamique 31

21 Première allocation dynamique 36
22 Copier un tableau 36
23 Chaînes dynamiques 37
24 Filtrer dans un tableau dynamique 38
25 Tableau extensible avec realloc 39
26 Une structure entièrement dynamique 40
27 Matrice dynamique 41
28 Inventaire extensible 41
29 Liste chaînée 42
30 Projet final : gestionnaire de classe 44

A ANNEXES
Aide-mémoire, erreurs classiques et outils 46

NIVEAU 1 · FACILE

Mémoire et pointeurs

Adresses, pointeurs, passage par adresse

CHAPITRE 3 DU COURS · ≈ 2 H 10 DE PRATIQUE

Objectifs du niveau

- Comprendre la notion d'adresse mémoire
- Déclarer, initialiser et déréférencer un pointeur
- Modifier une variable depuis une fonction
- Renvoyer plusieurs résultats grâce aux pointeurs
- Sécuriser son code avec `NULL`

Les dix exercices

01	Observer des adresses	10 min
02	Déclarer, initialiser, déréférencer	10 min
03	Prédire avant d'exécuter	15 min
04	Comparer des pointeurs ou des valeurs ?	10 min
05	Valeur ou adresse : la preuve par l'exemple	15 min
06	Élever au carré en toute sécurité	10 min
07	Échanger deux valeurs	15 min
08	Ordonner deux valeurs	10 min
09	Convertir une durée	20 min
10	Rotation de trois valeurs	15 min

FICHE THÉORIQUE 1

Mémoire et pointeurs

Avant les exercices 01 à 10

1.1 La mémoire : une longue rangée de cases numérotées

La mémoire vive (RAM) est une suite d'**octets**. Chaque octet possède un numéro unique : son **adresse**. Quand tu déclares une variable, le compilateur lui réserve un emplacement d'une taille qui dépend de son **type**.

Une variable possède…	Exemple pour `int age = 25;`	Comment l'obtenir ?
un **nom**	`age`	c'est toi qui le choisis
un **type**	`int` (en général 4 octets)	`sizeof(int)` ou `sizeof age`
une **valeur**	`25`	`age`
une **adresse**	par ex. `0x7ffd1004`	`&age`

Rappel

L'adresse affichée change d'une exécution à l'autre : c'est normal. Ce qui compte, ce n'est pas la valeur exacte de l'adresse, mais **la relation** entre une variable et son adresse.

1.2 Le pointeur : une variable qui contient une adresse

Un **pointeur** est une variable comme une autre… sauf que sa valeur est l'adresse d'une autre variable. On dit qu'il *pointe vers* cette variable. Son type précise le type de la donnée pointée : un `int *` pointe vers un `int`.

pointeur.c

```c
int age = 25;
int *ptr = &age;          // ptr contient l'adresse de age

printf("%d\n", age);      // 25  -> lecture directe
printf("%d\n", *ptr);     // 25  -> lecture via le pointeur
*ptr = 26;                // modifie age à travers ptr
printf("%d\n", age);      // 26
```



*Le pointeur ptr contient l'adresse de age : *ptr donne accès à la valeur 25.*

1.3

Les opérateurs & et * : à bien distinguer

Écriture	Se lit	Résultat
`&age`	« adresse de age »	une adresse (type `int *`)
`int *ptr;`	« ptr est un pointeur vers int »	une **déclaration**
`*ptr`	« valeur pointée par ptr »	la variable située à l'adresse contenue dans ptr
`ptr = &b;`	« ptr pointe maintenant vers b »	le pointeur est **réaffecté**

⚠ **À ne pas confondre**

Le symbole `*` a deux rôles selon le contexte : dans une **déclaration** (`int *p;`) il signifie « p est un pointeur » ; dans une **instruction** (`*p = 5;`) il signifie « la case pointée par p ». `&` et `*` sont des opérations inverses : `*&x` vaut `x`.

1.4 Afficher des adresses et des tailles

affichage.c

```c
int nombre = 42;
int *p = &nombre;

printf("Adresse de nombre : %p\n", (void *)&nombre);
printf("Contenu de p      : %p\n", (void *)p);
printf("Taille d'un int   : %zu octets\n", sizeof(int));
```

- `%p` affiche une adresse ; la norme demande de la convertir en `(void *)`.
- `%zu` est le format adapté au résultat de `sizeof` (type `size_t`).
- Sur une machine 64 bits, un pointeur occupe généralement 8 octets, **quel que soit** le type pointé.

1.5 NULL et pointeurs non initialisés

Un pointeur qui n'a pas encore de cible valide doit valoir `NULL`. On peut alors le tester avant de s'en servir.

× Dangereux

```c
int *p;        // contient n'importe quoi
*p = 10;       // comportement indéfini !
```

✓ Sûr

```c
int *p = NULL;
if (p != NULL) {
    *p = 10;   // jamais exécuté ici
}
```

Attention

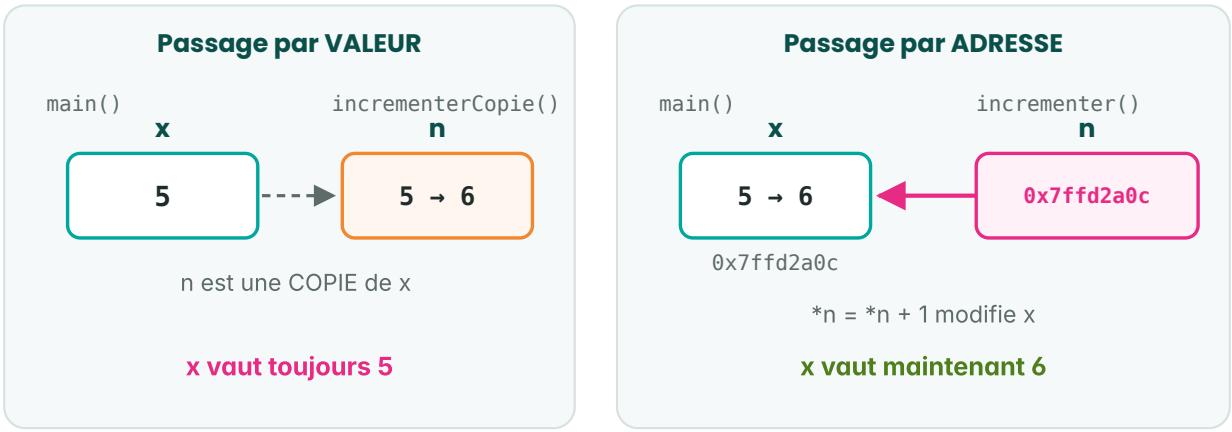
- Ne déréférence **jamais** un pointeur `NULL`, non initialisé ou devenu invalide.
- Un plantage (*segmentation fault*) signale presque toujours un de ces trois cas.

1.6 Passage par valeur, passage par adresse

En C, une fonction reçoit toujours une **copie** de ses arguments. Si elle reçoit la copie d'une valeur, elle ne peut pas modifier l'original. Si elle reçoit la copie d'une **adresse**, elle peut aller modifier la variable d'origine grâce à `*`.

```c
void incrementerCopie(int n) {
    n = n + 1;        // modifie la copie
}
```

```c
void incrementer(int *n) {
    *n = *n + 1;      // modifie l'original
}
```



À gauche, la fonction travaille sur une copie. À droite, elle reçoit l'adresse de x et le modifie réellement.

Rappel

Pour qu'une fonction modifie une variable de `main`, on lui passe **l'adresse** de cette variable : `incrementer(&x);`. Dans la fonction, on utilise `*n` pour atteindre la variable.

1.7 Plusieurs résultats pour une seule fonction

Une fonction ne renvoie qu'une seule valeur avec `return`. Pour produire plusieurs résultats, on utilise des **paramètres de sortie** (des pointeurs). Convention courante : la valeur de retour indique le **succès** (1) ou **l'échec** (0).

sorties.c

```c
int diviser(int a, int b, int *quotient, int *reste) {
    if (b == 0) return 0;      // échec : on ne touche pas aux sorties
    *quotient = a / b;
    *reste    = a % b;
    return 1;                  // succès
}

int q, r;
if (diviser(17, 5, &q, &r)) printf("%d reste %d\n", q, r);   // 3 reste 2
```

À retenir

- `&x` donne l'adresse de x ; `*p` donne la variable pointée par p.
- Un pointeur s'initialise toujours : avec une adresse valide ou avec `NULL`.
- Pour modifier une variable dans une fonction, on transmet son adresse.
- Retour = succès ou échec ; paramètres pointeurs = résultats.

1 Exercices 01 à 10

Relis la fiche théorique 1 en cas de blocage : tous les outils nécessaires s'y trouvent.

EXERCICE 01

01 Observer des adresses

FACILE ●○○

10 min

NOTIONS `&` `%p` `sizeof`

Premier contact avec la mémoire : tu vas afficher la valeur, l'adresse et la taille de trois variables de types différents.

Ce que tu dois faire

1. Déclare `int entier = 42;`, `double reel = 3.14;` et `char lettre = 'C';`.
2. Pour chaque variable, affiche sa valeur, son adresse (`%p`) et sa taille (`sizeof`).
3. Déclare un pointeur `int *p = &entier;` puis affiche le contenu de `p` et la valeur `*p`.

>_ SORTIE ATTENDUE

```
entier : valeur = 42    adresse = 0x7ffd5c3e1a4c   taille = 4 octets
reel   : valeur = 3.14  adresse = 0x7ffd5c3e1a50   taille = 8 octets
lettre : valeur = C     adresse = 0x7ffd5c3e1a4b   taille = 1 octet
p contient 0x7ffd5c3e1a4c  ->  *p = 42
```

Les adresses varient d'une exécution à l'autre ; les tailles sont celles d'une machine 64 bits courante.

Conseil

Pour afficher une adresse proprement : `printf("%p", (void *)&entier);`. Vérifie que l'adresse contenue dans `p` est **identique** à `&entier`.

Question de réflexion

Que vaut `sizeof p` ? Pourquoi un `int *` et un `char *` ont-ils la même taille alors que `int` et `char` n'en ont pas ?

NIVEAU 1

02

EXERCICE 02

Déclarer, initialiser, déréférencer

FACILE ●○○

10 min

NOTIONS `int *` `*p` `affectation`

Un pointeur permet de **lire** et d'**écrire** une variable à distance. Tu vas le vérifier pas à pas.

Ce que tu dois faire

1. Déclare `int x = 10;` et un pointeur `p` initialisé avec l'adresse de `x`.
2. Sans jamais écrire `x =`, remplace la valeur de `x` par 25 en passant par `p`.
3. Déclare `int y = *p + 5;` puis affiche `x`, `*p` et `y`.

>_ SORTIE ATTENDUE

```
x  = 25
*p = 25
y  = 30
```

Rappel

`*p = 25;` signifie : « écris 25 dans la case dont l'adresse est stockée dans p ». La variable `x` change donc, même si son nom n'apparaît pas dans l'instruction.

03

EXERCICE 03

Prédire avant d'exécuter

FACILE ●○○

15 min

NOTIONS `réaffectation` `trace` `*q *= 2`

Un bon programmeur sait **simuler la mémoire** sur papier. Sans exécuter le code, complète le tableau ligne par ligne, puis vérifie en compilant.

trace.c

```c
int a = 3, b = 7;
int *p = &a, *q = &b;      // ligne 2
*p = *q + 1;               // ligne 3
q = p;                     // ligne 4
*q *= 2;                   // ligne 5
printf("%d %d %d %d\n", a, b, *p, *q);
```

Après la ligne	a	b	p pointe vers	q pointe vers
2	3	7	a	b
3				
4				
5				

Conseil

Dessine deux boîtes (`a` , `b`) et deux flèches (`p` , `q`). À chaque ligne, demande-toi : *est-ce que je modifie une **flèche** (le pointeur) ou une **boîte** (la valeur pointée) ?*

Question de réflexion

Après la ligne 4, que se passerait-il si on écrivait `*q = 0;` ? Quelle variable serait modifiée ?

EXERCICE 04

04 Comparer des pointeurs ou des valeurs ?

FACILE

10 min

NOTIONS `p == q` `*p == *q`

Comparer deux pointeurs et comparer les valeurs qu'ils désignent sont deux choses très différentes.

Ce que tu dois faire

1. Déclare `int x = 4, y = 4;` puis `int *p = &x, *q = &y;` .
2. Affiche le résultat de `p == q` puis celui de `*p == *q` .
3. Fais pointer `q` vers `x` , puis affiche à nouveau `p == q` .
4. Explique en une phrase la différence entre les deux comparaisons.

>_ SORTIE ATTENDUE

```
p == q   : 0
*p == *q : 1
Après q = &x -> p == q : 1
```

Rappel

`p == q` compare deux **adresses** : vrai seulement si les deux pointeurs désignent **la même** case. `*p == *q` compare les **valeurs** contenues dans les cases.

05

EXERCICE 05

Valeur ou adresse : la preuve par l'exemple

FACILE ●○○

15 min

NOTIONS `passage par valeur` `passage par adresse`

Tu vas écrire deux fonctions presque identiques et constater que seule l'une d'elles modifie réellement la variable du `main`.

prototypes

```
void incrementerCopie(int n);   // reçoit une copie de la valeur
void incrementer(int *n);       // reçoit l'adresse de la variable
```

Ce que tu dois faire

1. Implémente les deux fonctions : chacune ajoute 1 à son paramètre.
2. Dans `main`, déclare `int x = 5;`.
3. Appelle `incrementerCopie(x);` puis affiche `x`.
4. Appelle `incrementer(&x);` puis affiche `x`.

>_ SORTIE ATTENDUE

```
Après incrementerCopie : x = 5
Après incrementer      : x = 6
```

Conseil

Ajoute un `printf` **dans** `incrementerCopie` pour afficher `n` : tu verras que la copie vaut bien 6… mais qu'elle disparaît à la fin de la fonction.

Question de réflexion

Pourquoi l'appel `incrementer(x);` provoque-t-il un avertissement du compilateur ? Que contiendrait alors le paramètre `n` ?

06

EXERCICE 06

Élever au carré en toute sécurité

FACILE ●○○

10 min

NOTIONS `int *` `NULL` `code retour`

Écris une fonction qui remplace un entier par son carré. Elle doit **refuser** de travailler sur un pointeur `NULL`.

prototype

```
int carre(int *n);   // renvoie 1 si succès, 0 si n vaut NULL
```

Ce que tu dois faire

1. Si `n` vaut `NULL`, renvoie 0 sans rien faire.
2. Sinon, remplace `*n` par `*n * *n` et renvoie 1.
3. Teste avec 7, avec -3, puis avec `carre(NULL)`.

SORTIE ATTENDUE

```
7  -> 49   (retour : 1)
-3 -> 9    (retour : 1)
carre(NULL) -> retour : 0
```

Attention

Sans le test `if (n == NULL)`, l'appel `carre(NULL)` ferait planter le programme. **Une fonction qui reçoit un pointeur devrait toujours se demander s'il peut être NULL.**

07

EXERCICE 07

Échanger deux valeurs

FACILE

15 min

NOTIONS `swap` `variable temporaire`

C'est l'exercice incontournable sur les pointeurs : échanger le contenu de deux variables du `main`.

prototype

```
void echanger(int *a, int *b);
```

Ce que tu dois faire

1. Implémente `echanger` à l'aide d'une variable temporaire.
2. Teste avec `a = 3` et `b = 9` : affiche les valeurs avant et après.
3. Teste avec deux valeurs identiques (4 et 4).

SORTIE ATTENDUE

```
Avant : a = 3, b = 9
Après : a = 9, b = 3
Valeurs identiques : 4 et 4 -> 4 et 4
```

Conseil

Écris les trois lignes dans cet ordre : sauvegarder `*a`, copier `*b` dans `*a`, recopier la sauvegarde dans `*b`. Si tu commences par `*a = *b;`, la première valeur est perdue.

Question de réflexion

Pourquoi une fonction `void echanger(int a, int b)` ne pourra-t-elle **jamais** fonctionner, même avec une variable temporaire ?

08

EXERCICE 08

Ordonner deux valeurs

FACILE

10 min

NOTIONS `réutilisation` `pointeurs transmis`

Écris une fonction qui garantit qu'après l'appel, la première variable contient la plus petite valeur.

prototype

```c
void ordonner(int *a, int *b);   // après l'appel : *a <= *b
```

Ce que tu dois faire

1. Si `*a` est plus grand que `*b`, échange les deux valeurs.
2. **Réutilise** ta fonction `echanger` de l'exercice 07.
3. Teste avec (12, 5) puis avec (3, 8).

SORTIE ATTENDUE

```
ordonner(12, 5) -> 5 12
ordonner(3, 8)  -> 3 8
```

Rappel

Dans `ordonner`, `a` et `b` sont **déjà** des adresses. On écrit donc `echanger(a, b);` et non `echanger(&a, &b);`, qui transmettrait l'adresse des pointeurs eux-mêmes.

09

EXERCICE 09

Convertir une durée

FACILE

20 min

NOTIONS `paramètres de sortie` `/ et %` `validation`

Un chronomètre fournit une durée en secondes. Écris une fonction qui la décompose en heures, minutes et secondes.

prototype

```c
int convertirSecondes(int total, int *h, int *m, int *s);
```

NIVEAU 1

Ce que tu dois faire

1. Si `total` est négatif ou si un des pointeurs vaut `NULL`, renvoie 0 **sans modifier** les sorties.
2. Sinon, calcule `*h`, `*m` et `*s`, puis renvoie 1.
3. Teste avec 3725, avec 59, puis avec -5.

SORTIE ATTENDUE

```
3725 s -> 1 h 2 min 5 s
59 s   -> 0 h 0 min 59 s
-5 s   -> durée invalide
```

Conseil

Une heure = 3600 secondes. Les minutes restantes s'obtiennent avec `(total % 3600) / 60`, et les secondes avec `total % 60`.

Question de réflexion

Quelles données peuvent être modifiées par la fonction ? Pourquoi est-il préférable de ne rien modifier en cas d'entrée invalide ?

10

EXERCICE 10

Rotation de trois valeurs

FACILE

15 min

NOTIONS `synthèse` `3 pointeurs`

Exercice de synthèse du niveau 1 : faire « tourner » trois variables vers la gauche.

prototype

```c
void rotation(int *a, int *b, int *c);   // a <- b, b <- c, c <- ancien a
```

Ce que tu dois faire

1. Implémente la rotation avec **une seule** variable temporaire.
2. Teste avec (1, 2, 3) : affiche le résultat après un appel.
3. Appelle `rotation` trois fois de suite : que constates-tu ?

SORTIE ATTENDUE

```
Départ          : 1 2 3
Après 1 rotation : 2 3 1
Après 3 rotations: 1 2 3
```

Conseil

Fais d'abord un schéma : trois boîtes et des flèches qui montrent dans quel sens les valeurs se déplacent. La seule valeur « en danger » est celle de `*a`.

Pour aller plus loin

Écris `rotationDroite` (a ← c, b ← a, c ← b) en **réutilisant** uniquement `rotation`. Combien d'appels faut-il ?

Auto-évaluation — niveau 1

- [ ] Je sais expliquer la différence entre une valeur et une adresse.
- [ ] Je sais déclarer, initialiser et déréférencer un pointeur.
- [ ] Je distingue le `*` de déclaration du `*` de déréférencement.
- [ ] Je sais afficher une adresse avec `%p` et une taille avec `%zu`.
- [ ] Je sais écrire une fonction qui modifie une variable du `main`.
- [ ] Je teste systématiquement les pointeurs `NULL` reçus en paramètre.

NIVEAU 2 · MODÉRÉ

Structures et organisation des données

struct, tableaux de structures

CHAPITRE 4 DU COURS · ≈ 3 H 30 DE PRATIQUE

Objectifs du niveau

- Relier tableaux et arithmétique des pointeurs
- Définir des structures avec `typedef`
- Manipuler une structure via un pointeur et `->`
- Parcourir, rechercher et trier un tableau de structures
- Organiser des données imbriquées

Les dix exercices

11	Parcourir un tableau avec un pointeur	15 min
12	Inverser un tableau avec deux pointeurs	20 min
13	Rechercher et renvoyer une adresse	20 min
14	Première structure : des points dans le plan	15 min
15	Fiche étudiant	20 min
16	Catalogue : un tableau de structures	20 min
17	Trouver un produit par son nom	20 min
18	Statistiques sur le catalogue	20 min
19	Trier le catalogue par prix	25 min
20	Mini-projet : la bibliothèque	35 min

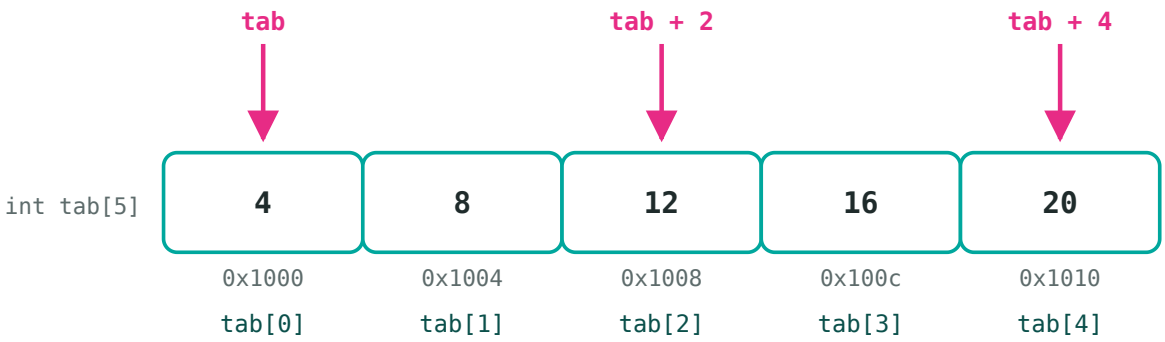
FICHE THÉORIQUE 2

Structures et organisation des données

Avant les exercices 11 à 20

2.1 Tableaux et pointeurs : deux faces d'une même mémoire

Les éléments d'un tableau sont rangés dans des cases **contiguës**. Dans la plupart des expressions, le nom du tableau se convertit en un pointeur vers son **premier élément** : `tab` vaut `&tab[0]`.



Cases contiguës : chaque int occupe 4 octets → l'adresse avance de 4

tab + i désigne l'adresse de l'élément i : le compilateur avance de i × sizeof(int) octets.

Notation tableau	Notation pointeur	Signification
`tab[i]`	`*(tab + i)`	valeur de l'élément d'indice i
`&tab[i]`	`tab + i`	adresse de l'élément d'indice i
`tab[0]`	`*tab`	premier élément
—	`p2 - p1`	nombre d'éléments entre deux pointeurs (type `ptrdiff_t`, format `%td`)

Rappel

L'arithmétique des pointeurs compte en **éléments**, pas en octets : si `p` est un `int *`, `p + 1` avance de `sizeof(int)` octets.

Passer un tableau à une fonction

Une fonction ne reçoit jamais le tableau entier, seulement l'adresse de son premier élément. Elle ignore donc sa taille : il faut **toujours** la transmettre en paramètre. Le mot-clé `const` indique que la fonction ne modifiera pas les éléments.

somme.c

```c
int somme(const int *tab, int taille) {
    int total = 0;
    for (const int *p = tab; p < tab + taille; p++) {
        total += *p;
    }
    return total;
}
```

> **Attention**
> Dans une fonction, `sizeof tab` donne la taille d'un **pointeur** (8 octets), pas celle du tableau ! Ne l'utilise jamais pour calculer le nombre d'éléments d'un paramètre.

2.2 Définir et utiliser une structure

Une **structure** regroupe plusieurs données de types différents sous un même nom. Avec `typedef`, on lui donne un nom de type court et lisible.

produit.c

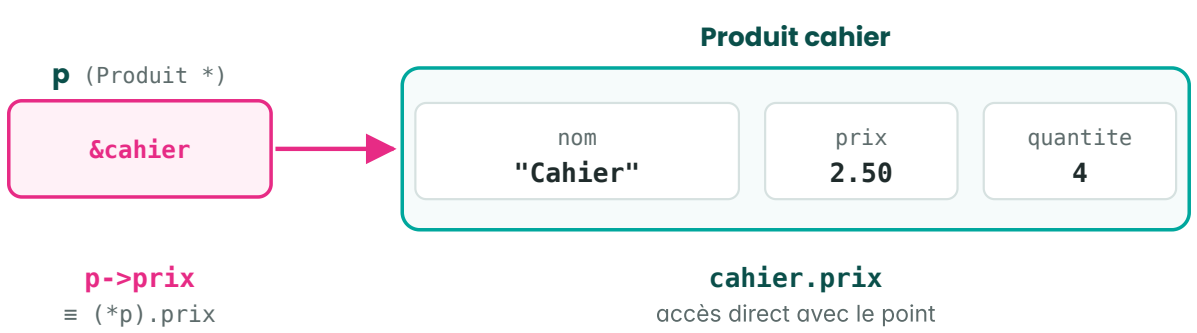
```c
typedef struct {
    char  nom[50];
    float prix;
    int   quantite;
} Produit;

Produit cahier = {"Cahier", 2.50f, 4};      // initialisation
Produit stylo  = {.nom = "Stylo", .prix = 1.20f, .quantite = 10};

printf("%s : %.2f EUR\n", cahier.nom, cahier.prix);   // accès avec le point
cahier.quantite += 2;
```

2.3 Pointeur vers une structure : l'opérateur ->

Quand on dispose d'un **pointeur** vers une structure, on accède à ses champs avec la flèche `->`. L'écriture `p->prix` est un raccourci pour `(*p).prix` (les parenthèses sont obligatoires, car `.` est prioritaire sur `*`).



Avec la variable : le point. Avec un pointeur : la flèche.

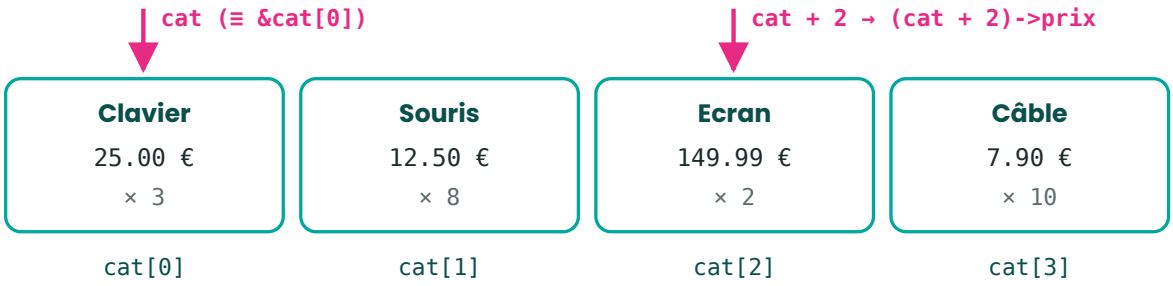
Pourquoi passer une structure par pointeur ?	Exemple de prototype
Pour la **modifier** dans la fonction	`void appliquerRemise(Produit *p, float remise);`
Pour éviter de **copier** une grosse structure	`void afficher(const Produit *p);`
Pour **renvoyer** l'adresse d'un élément trouvé	`Produit *trouver(Produit *cat, int n, const char *nom);`

Rappel

`const Produit *p` = « je promets de ne pas modifier le produit pointé ». C'est le bon réflexe pour toute fonction qui ne fait que **lire**.

2.4 Tableaux de structures

Un tableau de structures est la forme la plus naturelle pour représenter un **catalogue**, une **classe**, un **inventaire**… Chaque case contient une structure complète.



cat[i].prix et (cat + i)→prix désignent le même champ.

catalogue.c

```c
Produit cat[3] = {{"Cahier", 2.50f, 4}, {"Stylo", 1.20f, 10}, {"Classeur", 4.00f, 2}};

for (int i = 0; i < 3; i++)                    // version indice
    printf("%s\n", cat[i].nom);

for (const Produit *p = cat; p < cat + 3; p++)   // version pointeur
    printf("%s\n", p->nom);
```

2.5 Structures imbriquées et chaînes de caractères

Un champ peut lui-même être une structure. On enchaîne alors les accès : `livre.retour.jour` ou `p->retour.jour`.

imbrication.c

```c
typedef struct { int jour, mois, annee; } Date;
typedef struct { char titre[60]; Date retour; } Livre;

Livre l = {"Dune", {20, 11, 2026}};
Livre *p = &l;
printf("%02d/%02d\n", p->retour.jour, p->retour.mois);
```

NIVEAU 2

Attention

- On ne compare **pas** deux chaînes avec `==` (cela compare deux adresses) : on utilise `strcmp(a, b) == 0`.
- On ne copie pas une chaîne avec `=` dans un tableau de `char` : on utilise `strcpy` ou, plus sûr, `snprintf`.
- N'oublie pas `#include <string.h>`.

À retenir

- `tab[i]` ≡ `*(tab + i)` ; une fonction qui reçoit un tableau a besoin de sa taille.
- Variable structure → `.` ; pointeur vers structure → `->`.
- Fonction qui lit : `const Type *` ; fonction qui modifie : `Type *`.
- Chaînes : `strcmp` pour comparer, `strcpy` / `snprintf` pour copier.

2 Exercices 11 à 20

Relis la fiche théorique 2 en cas de blocage : tous les outils nécessaires s'y trouvent.

11 EXERCICE 11 — Parcourir un tableau avec un pointeur

MODÉRÉ ●●○ — 15 min

NOTIONS `p++` `tab + taille` `const`

Affiche les éléments d'un tableau **sans utiliser d'indice** : un pointeur se déplace de case en case.

prototypes

```c
void afficherTableau(const int *tab, int taille);
void afficherALEnvers(const int *tab, int taille);
```

Ce que tu dois faire

1. Dans `afficherTableau`, utilise une boucle `for (const int *p = tab; p < tab + taille; p++)`.
2. Écris `afficherALEnvers` en partant de `tab + taille - 1` et en reculant.
3. Teste avec `{3, 1, 4, 1, 5}`. Aucune écriture `tab[i]` n'est autorisée.

SORTIE ATTENDUE

```
Endroit : 3 1 4 1 5
Envers  : 5 1 4 1 3
```

Conseil

`tab + taille` est l'adresse **juste après** le dernier élément : on peut la calculer et la comparer, mais jamais la déréférencer.

12 EXERCICE 12 — Inverser un tableau avec deux pointeurs

MODÉRÉ ●●○ — 20 min

NOTIONS `deux pointeurs` `comparaison d'adresses`

Inverse un tableau **sur place** : un pointeur part du début, l'autre de la fin, et ils échangent leurs valeurs jusqu'à se croiser.

prototype

```c
void inverserTableau(int *tab, int taille);
```

Ce que tu dois faire

1. Initialise `int *debut = tab;` et `int *fin = tab + taille - 1;`.
2. Tant que `debut < fin` : échange `*debut` et `*fin`, puis rapproche les deux pointeurs.
3. Teste avec une taille impaire `{1, 2, 3, 4, 5}` et une taille paire `{1, 2, 3, 4}`.

>_ SORTIE ATTENDUE

```
{1, 2, 3, 4, 5} -> 5 4 3 2 1
{1, 2, 3, 4}    -> 4 3 2 1
```

Rappel

Deux pointeurs vers le **même tableau** peuvent être comparés avec `<` : le plus petit désigne l'élément situé le plus à gauche.

Question de réflexion

Que se passe-t-il si la condition est `debut != fin` au lieu de `debut < fin` avec un tableau de taille paire ?

NIVEAU 2

13 EXERCICE 13 — Rechercher et renvoyer une adresse

MODÉRÉ ●●○

20 min

NOTIONS `retour int *` `NULL` `p - tab`

Une fonction peut renvoyer un **pointeur** vers l'élément trouvé, ou `NULL` s'il n'existe pas. L'appelant peut ensuite le lire… ou le modifier directement.

prototype

```c
int *rechercher(int *tab, int taille, int valeur);
```

Ce que tu dois faire

1. Parcours le tableau avec un pointeur et renvoie l'adresse de la **première** occurrence de `valeur`.
2. Renvoie `NULL` si la valeur est absente.
3. Dans `main`, calcule l'indice trouvé avec `resultat - tab` et affiche-le avec `%td`.
4. Teste avec `{7, 3, 9, 3}` : recherche 9, 3 puis 4. Remplace ensuite le 9 par 0 **via le pointeur renvoyé**.

>_ SORTIE ATTENDUE

```
9 trouvé à l'indice 2
3 trouvé à l'indice 1
4 introuvable
Tableau modifié : 7 3 0 3
```

Attention

Teste **toujours** le résultat avant de le déréférencer : `if (r != NULL) *r = 0;`.

14

EXERCICE 14

Première structure : des points dans le plan

MODÉRÉ ●●○

15 min

NOTIONS `typedef struct` `->` `const`

Modélise un point du plan et écris deux fonctions : l'une le **modifie**, l'autre se contente de le **lire**.

point.h

```c
typedef struct {
    double x;
    double y;
} Point;

void   deplacer(Point *p, double dx, double dy);
double distance(const Point *a, const Point *b);
```

Ce que tu dois faire

1. Implémente `deplacer` (ajoute dx et dy aux coordonnées) avec l'opérateur `->`.
2. Implémente `distance` avec `sqrt` (inclure `math.h`, compiler avec `-lm`).
3. Crée `A(1 ; 2)` et l'origine `O(0 ; 0)`. Déplace A de (+2 ; +2) puis affiche A et la distance OA.

>_ SORTIE ATTENDUE

```
A après déplacement : (3.00 ; 4.00)
Distance OA : 5.00
```

Conseil

Observe les prototypes : `deplacer` reçoit `Point *` car il modifie le point ; `distance` reçoit `const Point *` car il ne fait que lire. Le compilateur refusera toute modification accidentelle dans `distance`.

NIVEAU 2

15

EXERCICE 15

Fiche étudiant

MODÉRÉ ●●○

20 min

NOTIONS `tableau dans une struct` `char[]` `moyenne`

Une structure peut contenir des tableaux : un nom (chaîne) et un tableau de notes.

etudiant.h

```c
typedef struct {
    char   nom[50];
    int    age;
    double notes[3];
} Etudiant;

void   afficherEtudiant(const Etudiant *e);
double moyenneEtudiant(const Etudiant *e);
```

Ce que tu dois faire

1. Crée l'étudiante `{"Alice", 20, {14, 16, 12}}`.
2. `afficherEtudiant` affiche le nom, l'âge et les trois notes.
3. `moyenneEtudiant` calcule la moyenne des notes.
4. Écris une fonction `void ajouterPoint(Etudiant *e, int indice)` qui ajoute 1 point à une note sans dépasser 20.

SORTIE ATTENDUE

```
Alice (20 ans) : 14.0  16.0  12.0
Moyenne : 14.00
```

Rappel

Avec un pointeur, on accède à une case du tableau interne par `e->notes[i]`. La flèche s'applique à la structure, les crochets au tableau qu'elle contient.

Question de réflexion

Que doit faire `ajouterPoint` si `indice` vaut 5 ? Comment le signaler à l'appelant ?

NIVEAU 2

16

EXERCICE 16

Catalogue : un tableau de structures

MODÉRÉ ●●●

20 min

NOTIONS `Produit cat[]` `parcours` `total`

Voici le catalogue qui servira pour les exercices 16 à 19. Recopie-le dans ton programme.

donnees.c

```c
Produit cat[4] = {
    {"Clavier",    25.00f,  3},
    {"Souris",     12.50f,  8},
    {"Ecran",     149.99f,  2},
    {"Cable HDMI",  7.90f, 10}
};
```

Ce que tu dois faire

1. Écris `void afficherCatalogue(const Produit *cat, int n)` qui affiche chaque produit sur une ligne.
2. Écris `double valeurStock(const Produit *cat, int n)` qui renvoie la somme des `prix × quantite`.
3. Parcours le tableau **avec un pointeur** dans au moins une des deux fonctions.

SORTIE ATTENDUE

```
Clavier         25.00 EUR  x 3
Souris          12.50 EUR  x 8
Ecran          149.99 EUR  x 2
Cable HDMI       7.90 EUR  x 10
Valeur totale du stock : 553.98 EUR
```

Conseil

Pour aligner l'affichage, utilise des largeurs fixes, par exemple le format `%-12s %8.2f EUR x %d` : `%-12s` aligne le texte à gauche sur 12 caractères, `%8.2f` affiche le nombre sur 8 caractères avec 2 décimales.

17

EXERCICE 17

Trouver un produit par son nom

MODÉRÉ ●●●

20 min

NOTIONS `strcmp` `retour Produit *` `modification`

Recherche un produit dans le catalogue et modifie-le **directement** grâce au pointeur renvoyé.

prototype

```c
Produit *trouverProduit(Produit *cat, int n, const char *nom);
```

NIVEAU 2

Ce que tu dois faire

1. Compare les noms avec `strcmp` ; renvoie `&cat[i]` si trouvé, `NULL` sinon.
2. Dans `main`, cherche « Souris » et retire 3 unités du stock via le pointeur obtenu.
3. Affiche la nouvelle valeur du stock, puis cherche « Tablette ».

>_ SORTIE ATTENDUE

```
Souris trouvée : quantité 8 -> 5
Nouvelle valeur du stock : 516.48 EUR
Tablette : produit introuvable
```

Attention

`cat[i].nom == nom` compile, mais compare deux **adresses** : le résultat sera presque toujours faux. Utilise `strcmp(cat[i].nom, nom) == 0`.

EXERCICE 18

18 Statistiques sur le catalogue

MODÉRÉ ●●●

20 min

NOTIONS `const Produit *` `max` `comptage`

Analyse le catalogue de l'exercice 16 (valeurs d'origine) sans jamais le modifier.

prototypes

```c
const Produit *plusCher(const Produit *cat, int n);
int nombreEnAlerte(const Produit *cat, int n, int seuil);
```

Ce que tu dois faire

1. `plusCher` renvoie l'adresse du produit au prix le plus élevé (le premier en cas d'égalité).
2. `nombreEnAlerte` compte les produits dont la quantité est **strictement inférieure** au seuil.
3. Affiche le produit le plus cher puis le nombre de produits en alerte pour un seuil de 5.

>_ SORTIE ATTENDUE

```
Produit le plus cher : Ecran (149.99 EUR)
Produits sous le seuil de 5 : 2
```

Conseil

Commence avec `const Produit *max = cat;` (le premier élément) puis compare chaque produit suivant à `max->prix`. Pas besoin de copier les structures : on ne manipule que des adresses.

NIVEAU 2

Question de réflexion

Que doit renvoyer `plusCher` si `n` vaut 0 ? Modifie ta fonction pour gérer ce cas.

EXERCICE 19

19 Trier le catalogue par prix

MODÉRÉ ●●●

25 min

NOTIONS `tri à bulles` `échange de structures`

Trie le catalogue par **prix croissant** en échangeant des structures entières.

prototypes

```
void echangerProduits(Produit *a, Produit *b);
void trierParPrix(Produit *cat, int n);
```

Ce que tu dois faire

1. `echangerProduits` utilise une variable temporaire de type `Produit` (l'affectation copie tous les champs).
2. `trierParPrix` implémente un tri à bulles qui appelle `echangerProduits(&cat[j], &cat[j + 1])`.
3. Affiche le catalogue trié.

SORTIE ATTENDUE

```
Cable HDMI       7.90 EUR
Souris          12.50 EUR
Clavier         25.00 EUR
Ecran          149.99 EUR
```

Rappel

Contrairement aux tableaux, les structures **peuvent** être copiées avec `=` : `Produit tmp = *a;` copie aussi le tableau `nom` qu'elle contient.

Pour aller plus loin

Ajoute un paramètre `int croissant` pour choisir l'ordre du tri, puis écris un tri **par nom** avec `strcmp`.

NIVEAU 2

EXERCICE 20

20 Mini-projet : la bibliothèque

MODÉRÉ

35 min

NOTIONS `structures imbriquées` `comparaison de dates` `synthèse`

Synthèse du niveau 2. Une bibliothèque veut connaître les livres **empruntés** dont la date de retour est **dépassée**.

bibliotheque.h

```c
typedef struct { int jour, mois, annee; } Date;

typedef struct {
    char titre[60];
    char auteur[40];
    Date retour;
    int  emprunte;      // 1 = emprunté, 0 = disponible
} Livre;

int comparerDates(const Date *a, const Date *b);   // -1 si a<b, 0 si égales, 1 si a>b
int livresEnRetard(const Livre *biblio, int n, const Date *aujourdhui);
```

donnees.c

```c
Livre biblio[5] = {
    {"Le Petit Prince", "Saint-Exupery",       {10, 11, 2026}, 1},
    {"1984",            "Orwell",              {20, 11, 2026}, 1},
    {"Dune",            "Herbert",             { 1, 10, 2026}, 0},
    {"Le langage C",    "Kernighan & Ritchie", {14, 11, 2026}, 1},
    {"Fondation",       "Asimov",              {15, 11, 2026}, 1}
};
Date aujourdhui = {15, 11, 2026};
```

Ce que tu dois faire

1. `comparerDates` compare d'abord les années, puis les mois, puis les jours.
2. `livresEnRetard` affiche chaque livre en retard et renvoie leur nombre.
3. Un livre **non emprunté** n'est jamais en retard ; un retour prévu **aujourd'hui** n'est pas un retard.

SORTIE ATTENDUE

```
Livres en retard au 15/11/2026 :
  - Le Petit Prince (Saint-Exupery), retour prévu le 10/11/2026
  - Le langage C (Kernighan & Ritchie), retour prévu le 14/11/2026
Total : 2 livre(s) en retard
```

Conseil

Avance par petites étapes : teste `comparerDates` seule avec trois cas (avant, égal, après) **avant** d'écrire `livresEnRetard`.

NIVEAU 2

Question de réflexion

« Dune » a une date dépassée : pourquoi n'apparaît-il pas ? Et « Fondation » ? Vérifie que ton programme traite correctement ces deux cas limites.

✓ Auto-évaluation — niveau 2

- [ ] Je sais parcourir un tableau avec un pointeur, sans indice.
- [ ] Je sais pourquoi une fonction doit recevoir la taille d'un tableau.
- [ ] Je sais définir une structure avec `typedef` et l'initialiser.
- [ ] J'utilise `.` avec une variable et `->` avec un pointeur.
- [ ] J'utilise `const` dans les fonctions qui ne font que lire.
- [ ] Je compare des chaînes avec `strcmp`, jamais avec `==`.

NIVEAU 3 · EXPERT

Allocation dynamique

malloc, realloc, free

CHAPITRE 5 DU COURS · ≈ 5 H 15 DE PRATIQUE

Objectifs du niveau

- Distinguer la pile et le tas
- Réserver et libérer la mémoire sans fuite
- Faire grandir un tableau avec `realloc`
- Modifier un pointeur grâce à `Type **`
- Construire des structures de données dynamiques

Les dix exercices

21	Première allocation dynamique	20 min
22	Copier un tableau	20 min
23	Chaînes dynamiques	25 min
24	Filtrer dans un tableau dynamique	25 min
25	Tableau extensible avec realloc	30 min
26	Une structure entièrement dynamique	30 min
27	Matrice dynamique	30 min
28	Inventaire extensible	35 min
29	Liste chaînée	40 min
30	Projet final : gestionnaire de classe	60 min

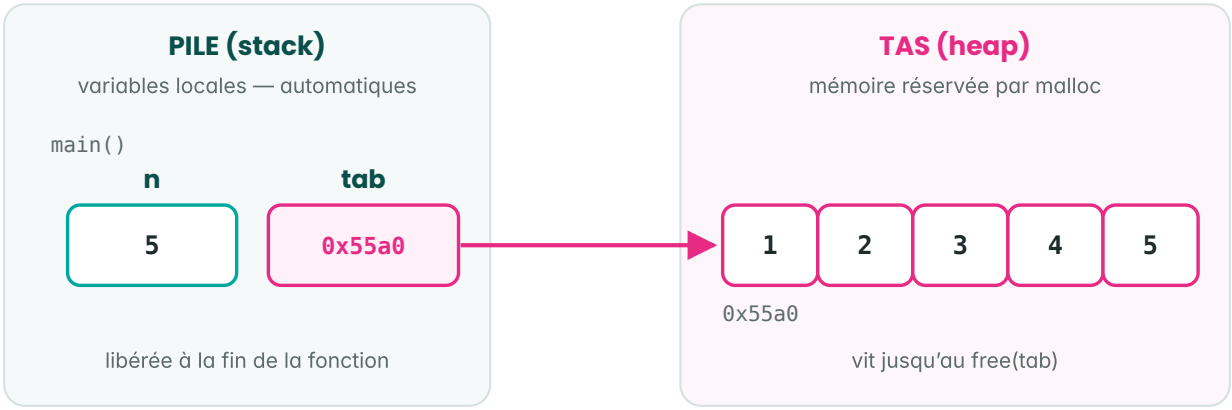
FICHE THÉORIQUE 3

Allocation dynamique

Avant les exercices 21 à 30

3.1 Pile et tas : deux zones, deux durées de vie

Les variables locales vivent sur la **pile** : elles sont créées à l'entrée de la fonction et détruites automatiquement à sa sortie. Leur taille doit être connue à la compilation. Le **tas** est une réserve de mémoire que le programme demande **pendant l'exécution**, et qu'il doit rendre lui-même.



Le pointeur tab vit sur la pile ; le bloc qu'il désigne vit sur le tas jusqu'au free.

On utilise le tas quand…	Exemple
la taille n'est connue qu'à l'exécution	l'utilisateur choisit le nombre d'éléments
la taille évolue pendant le programme	un inventaire auquel on ajoute des produits
la donnée doit survivre à la fonction qui la crée	une fonction `creerEtudiant` qui renvoie un étudiant

3.2 Réserver : malloc et calloc

allocation.c

```c
#include <stdlib.h>

int *tab = malloc(n * sizeof *tab);     // n entiers NON initialisés
if (tab == NULL) {                      // l'allocation peut échouer
    fprintf(stderr, "Mémoire insuffisante\n");
    return EXIT_FAILURE;
}

double *notes = calloc(n, sizeof *notes);   // n doubles initialisés à 0
```

Fonction	Rôle	Contenu du bloc
`malloc(taille)`	réserve `taille` octets	indéterminé (valeurs « au hasard »)
`calloc(n, taille)`	réserve `n` éléments de `taille` octets	mis à zéro
`realloc(p, taille)`	redimensionne le bloc `p`	conservé (jusqu'à l'ancienne taille)
`free(p)`	rend le bloc au système	—

Rappel

L'idiome `sizeof *tab` (plutôt que `sizeof(int)`) reste correct même si tu changes le type de `tab` plus tard. Et **tout** `malloc` est suivi d'un test `== NULL`.

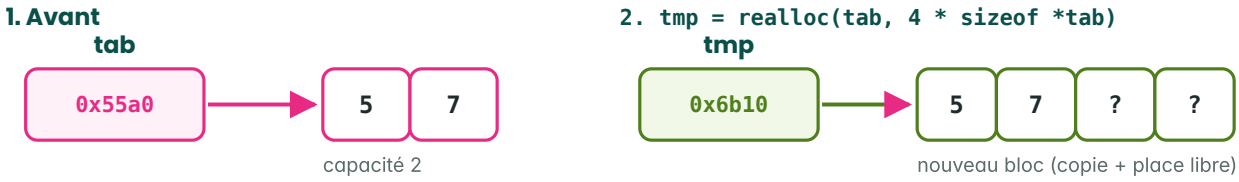
3.3 Libérer : free et ses règles d'or

Les quatre règles d'or

- **Un malloc = un free.** Un bloc jamais libéré est une *fuite mémoire*.
- Après `free(p)`, le bloc n'existe plus : utiliser `*p` est interdit (*use after free*). Réflexe : `p = NULL;`.
- Ne libère **jamais** deux fois le même bloc (*double free*), ni un tableau déclaré sur la pile.
- `free(NULL)` est autorisé et ne fait rien.

3.4 Agrandir : realloc et le pointeur temporaire

`realloc` peut agrandir le bloc sur place… ou en réserver un nouveau ailleurs, y copier les données et libérer l'ancien. En cas d'échec, il renvoie `NULL` **sans libérer** l'ancien bloc : il faut donc passer par un pointeur temporaire.




```
3. Si tmp != NULL :   tab = tmp; → l'ancien bloc a déjà été libéré par realloc
Si tmp == NULL :      tab est TOUJOURS valide → on gère l'erreur, sans fuite
```

Le motif sûr : ne jamais écrire tab = realloc(tab, …) directement.

× Fuite possible

```
tab = realloc(tab, nouvelle);
// si échec : tab vaut NULL,
// l'ancien bloc est perdu
```

✓ Motif sûr

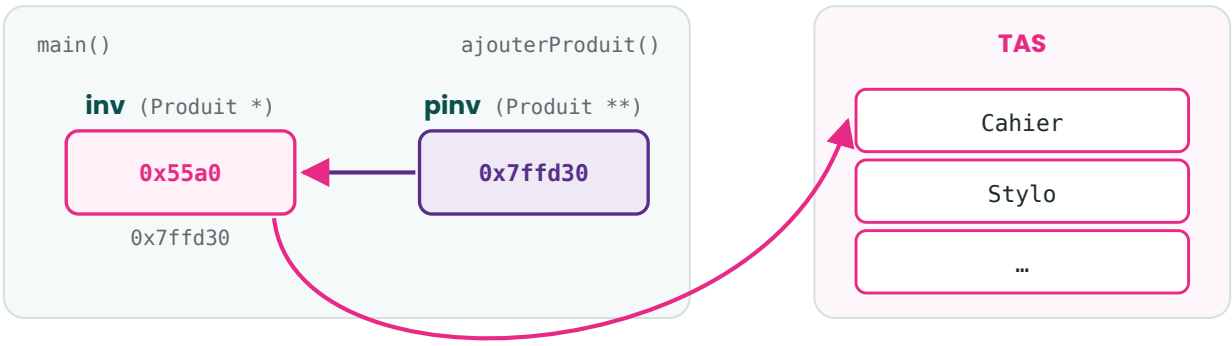
```
int *tmp = realloc(tab, nouvelle);
if (tmp == NULL) { /* erreur */ }
else tab = tmp;
```

Conseil

Pour un tableau qui grandit, **double la capacité** à chaque fois qu'il est plein plutôt que d'ajouter une case : on évite des centaines d'appels à `realloc`. Note : `realloc(NULL, t)` se comporte comme `malloc(t)`.

3.5 Modifier un pointeur dans une fonction : Type **

Pour qu'une fonction modifie un `int`, on lui passe un `int *`. Pour qu'elle modifie un **pointeur** (par exemple après un `realloc`), on lui passe **l'adresse du pointeur** : un `Type **`.



***pinv = tmp; ← modifie inv dans main()**

*pinv contient l'adresse de inv : *pinv = … modifie le pointeur du main.*

double_pointeur.c

```
int ajouterProduit(Produit **inv, int *n, int *capacite, Produit p);

Produit *inventaire = NULL;
int n = 0, capacite = 0;
ajouterProduit(&inventaire, &n, &capacite, nouveau);   // & devant chaque sortie
```

NIVEAU 3

3.6 Structures et mémoire dynamique

Quand une structure contient elle-même des pointeurs vers le tas, on libère **de l'intérieur vers l'extérieur** : d'abord les champs alloués, ensuite la structure.

liberation.c

```c
typedef struct { char *nom; double *notes; int nbNotes; } Etudiant;

free(e->nom);      // 1. les champs dynamiques
free(e->notes);
free(e);           // 2. la structure elle-même
e = NULL;
```

3.7 Détecter les erreurs mémoire

Les erreurs mémoire ne plantent pas toujours… ce qui les rend dangereuses. Deux outils les révèlent immédiatement :

terminal

```
# AddressSanitizer : intégré à GCC et Clang
gcc -Wall -Wextra -Wpedantic -g -fsanitize=address programme.c -o programme
./programme

# Valgrind (Linux) : bilan complet des fuites
valgrind --leak-check=full ./programme
```

>_ OBJECTIF À ATTEINDRE AVEC VALGRIND

```
==2841== HEAP SUMMARY:
==2841==     in use at exit: 0 bytes in 0 blocks
==2841==   total heap usage: 4 allocs, 4 frees, 1,124 bytes allocated
==2841== All heap blocks were freed -- no leaks are possible
```

À retenir

- `malloc` / `calloc` réservent, `realloc` redimensionne, `free` libère.
- Tester **chaque** allocation ; libérer **chaque** bloc, une seule fois.
- `realloc` passe toujours par un pointeur temporaire.
- Pour modifier un pointeur dans une fonction, on lui passe son adresse (`Type **`).

NIVEAU 3

3 Exercices 21 à 30

Relis la fiche théorique 3 en cas de blocage : tous les outils nécessaires s'y trouvent.

21 Première allocation dynamique

EXERCICE 21

EXPERT ●●●

20 min

NOTIONS `malloc` `free` `test NULL`

L'utilisateur choisit la taille du tableau : impossible de la fixer à la compilation.

Ce que tu dois faire

1. Demande un entier `n` compris entre 1 et 100 ; redemande tant que la saisie est invalide.
2. Alloue un tableau de `n` entiers avec `malloc` et vérifie le résultat.
3. Remplis-le avec les carrés `1, 4, 9, …, n²` puis affiche leur somme.
4. Libère la mémoire et mets le pointeur à `NULL`.

>_ SORTIE ATTENDUE

```
Taille du tableau (1-100) : 0
Valeur invalide.
Taille du tableau (1-100) : 5
Carrés : 1 4 9 16 25
Somme  : 55
```

Conseil

Contrôle aussi la valeur de retour de `scanf` : `if (scanf("%d", &n) != 1)` détecte une saisie qui n'est pas un nombre. Pour n = 100, la somme vaut 338350 : un bon test de ton programme.

22 Copier un tableau

EXERCICE 22

EXPERT ●●●

20 min

NOTIONS `retour int *` `propriété` `free par l'appelant`

Une fonction peut créer un tableau et le **renvoyer** : c'est alors à l'appelant de le libérer.

prototype

```c
int *copierTableau(const int *src, int n);   // NULL si erreur
```

Ce que tu dois faire

1. Si `src` vaut `NULL` ou si `n <= 0`, renvoie `NULL`.
2. Alloue un nouveau bloc, recopie les `n` valeurs et renvoie-le.
3. Dans `main`, copie `{1, 2, 3}`, remplace le premier élément de la **copie** par 99, puis affiche les deux tableaux.
4. N'oublie pas le `free` de la copie.

SORTIE ATTENDUE

```
Original : 1 2 3
Copie    : 99 2 3
```

Attention

Une fonction ne doit **jamais** renvoyer l'adresse d'un tableau local (`int t[10]; return t;`) : il est détruit à la sortie de la fonction. Seul un bloc alloué sur le tas survit.

Question de réflexion

Qui est « propriétaire » du bloc renvoyé ? Comment l'indiquer clairement dans un commentaire au-dessus du prototype ?

23

EXERCICE 23

Chaînes dynamiques

EXPERT ●●●

25 min

NOTIONS `strlen + 1` `\0` `memcpy / strcpy`

Écris tes propres fonctions de création de chaînes. La taille exacte de chaque bloc doit être calculée.

prototypes

```c
char *dupliquer(const char *s);                    // copie de s
char *concatener(const char *a, const char *b);    // nouvelle chaîne a + b
```

Ce que tu dois faire

1. `dupliquer` alloue `strlen(s) + 1` octets et y copie `s`.
2. `concatener` alloue exactement la place nécessaire pour `a`, `b` et le `\0` final.
3. Teste : `dupliquer("Bonjour")`, puis `concatener` avec `", ITLG"`.
4. Affiche le résultat, sa longueur, et libère **les deux** chaînes.

NIVEAU 3

>_ SORTIE ATTENDUE

```
Résultat  : Bonjour, ITLG
Longueur  : 13 caractères
Mémoire   : 14 octets réservés
```

Rappel

Une chaîne C se termine par le caractère `\0`. `strlen` ne le compte pas : il faut **toujours** prévoir `+ 1` octet lors de l'allocation.

EXERCICE 24

24 Filtrer dans un tableau dynamique

EXPERT ●●●

25 min

NOTIONS `taille calculée` `paramètre de sortie` `cas vide`

Extrais les nombres pairs d'un tableau dans un **nouveau** tableau de taille exacte.

prototype

```c
int *filtrerPairs(const int *tab, int n, int *nbResultats);
```

Ce que tu dois faire

1. Premier passage : compte les nombres pairs.
2. Alloue un tableau de cette taille exacte, puis remplis-le lors d'un second passage.
3. Écris le nombre de résultats dans `*nbResultats`.
4. S'il n'y a aucun nombre pair, renvoie `NULL` avec `*nbResultats = 0`.
5. Teste avec `{3, 8, 5, 12, 7, 4}` puis avec `{1, 3, 5}`.

>_ SORTIE ATTENDUE

```
{3, 8, 5, 12, 7, 4} -> 3 nombre(s) pair(s) : 8 12 4
{1, 3, 5}           -> aucun nombre pair
```

Conseil

Pourquoi deux passages ? Parce qu'on ne connaît pas la taille du résultat à l'avance. L'alternative avec `realloc` sera vue à l'exercice 25 : compare les deux approches.

NIVEAU 3

EXERCICE 25

25 Tableau extensible avec realloc

EXPERT ●●●

30 min

NOTIONS `realloc` `capacité × 2` `int **`

L'utilisateur saisit des entiers jusqu'à taper 0. On ne connaît pas leur nombre : le tableau doit **grandir** au besoin.

prototype

```
int ajouterValeur(int **tab, int *taille, int *capacite, int valeur);   // 1 si OK
```

Ce que tu dois faire

1. Démarre avec une capacité de 2 (allouée avec `malloc`).
2. Si le tableau est plein, **double** la capacité avec `realloc` (via un pointeur temporaire) et affiche l'évolution.
3. Après la saisie, affiche le nombre de valeurs, la capacité finale, la somme et la moyenne.
4. Libère le tableau.

>_ SORTIE ATTENDUE

```
Saisie : 5 7 2 9 4 0
  capacité 2 -> 4
  capacité 4 -> 8
Valeurs : 5   Capacité : 8
Somme : 27   Moyenne : 5.40
```

Rappel

Dans la fonction, `*tab` est le pointeur du `main`, et `(*tab)[i]` est la case d'indice i. Les parenthèses sont **obligatoires** : `*tab[i]` signifierait `*(tab[i])`.

Question de réflexion

Que se passe-t-il pour les valeurs déjà saisies si `realloc` échoue ? Ton programme les perd-il ?

NIVEAU 3

26

EXERCICE 26

Une structure entièrement dynamique

EXPERT ●●●

30 min

NOTIONS `malloc de struct` `libération en ordre` `Type **`

Un étudiant est maintenant créé sur le tas, avec un nom et un tableau de notes **eux aussi** dynamiques.

etudiant_dyn.h

```
typedef struct {
    char   *nom;
    double *notes;
    int     nbNotes;
} Etudiant;

Etudiant *creerEtudiant(const char *nom, int nbNotes);
void      detruireEtudiant(Etudiant **e);
```

Ce que tu dois faire

1. `creerEtudiant` alloue la structure, le nom (réutilise `dupliquer`) et les notes (avec `calloc`).
2. Si **une** des allocations échoue, libère ce qui a déjà été réservé et renvoie `NULL`.
3. `detruireEtudiant` libère tout dans le bon ordre puis met le pointeur de l'appelant à `NULL`.
4. Crée Alice avec 3 notes (14, 16, 12), affiche sa moyenne, détruis-la et vérifie que le pointeur vaut `NULL`.

>_ SORTIE ATTENDUE

```
Alice : 14.0 16.0 12.0 -> moyenne 14.00
Après destruction : pointeur = NULL
```

Conseil

`detruireEtudiant` reçoit un `Etudiant **` pour une seule raison : pouvoir écrire `*e = NULL;` et protéger l'appelant contre une réutilisation accidentelle.

NIVEAU 3

27

EXERCICE 27

Matrice dynamique

EXPERT ●●●

30 min

NOTIONS `int **` `tableau de pointeurs` `libération en boucle`

Construis une matrice dont le nombre de lignes et de colonnes est choisi à l'exécution : un tableau de pointeurs, chacun vers une ligne.

prototypes

```
int **creerMatrice(int lignes, int colonnes);
void  libererMatrice(int ***m, int lignes);
```

Ce que tu dois faire

1. Alloue le tableau de `lignes` pointeurs, puis chaque ligne de `colonnes` entiers.
2. Si une ligne échoue, libère les lignes déjà créées **et** le tableau de pointeurs.
3. Remplis une matrice 3 × 4 avec `m[i][j] = (i + 1) * (j + 1)` et affiche-la.
4. Libère chaque ligne, puis le tableau de pointeurs, puis mets `*m` à `NULL`.

SORTIE ATTENDUE

```
1   2   3   4
2   4   6   8
3   6   9   12
```

Rappel

Nombre de `free` à effectuer : **une par ligne + une pour le tableau de pointeurs**, soit 4 pour une matrice 3 × 4. Libérer d'abord le tableau de pointeurs rendrait les lignes inaccessibles : fuite assurée.

28

EXERCICE 28

Inventaire extensible

EXPERT ●●●

35 min

NOTIONS `Produit **` `realloc` `suppression`

Version avancée de l'inventaire : on peut ajouter **et** supprimer des produits. Le tableau s'adapte dans les deux sens.

inventaire.h

```
int ajouterProduit(Produit **inv, int *n, int *capacite, Produit p);
int supprimerProduit(Produit **inv, int *n, int index);
double valeurTotale(const Produit *inv, int n);
```

NIVEAU 3

Ce que tu dois faire

1. Démarre avec `Produit *inv = NULL;`, `n = 0`, `capacite = 0`.
2. `ajouterProduit` : si plein, capacité = 2 (au départ) puis × 2, via `realloc`.
3. `supprimerProduit` : vérifie l'indice, décale les éléments suivants vers la gauche, décrémente `n`.
4. Ajoute Cahier (2.50 × 4), Stylo (1.20 × 10), Classeur (4.00 × 2), Agenda (8.90 × 1) ; affiche la valeur totale.
5. Supprime le Stylo (indice 1), affiche à nouveau, puis teste la suppression de l'indice 7.

SORTIE ATTENDUE

```
4 produits (capacité 4) - valeur totale : 38.90 EUR
Suppression de Stylo...
3 produits : Cahier, Classeur, Agenda - valeur totale : 26.90 EUR
Suppression de l'indice 7 : indice invalide
```

Conseil

Dans la fonction, écris `Produit *t = *inv;` au début pour alléger les accès : `t[i]` est plus lisible que `(*inv)[i]`. Mais **après** un `realloc`, c'est bien `*inv` qu'il faut mettre à jour.

Question de réflexion

Quelles données peuvent être modifiées par chaque fonction ? Que se passe-t-il si l'indice de suppression est invalide ?

NIVEAU 3

29 EXERCICE 29 — Liste chaînée

EXPERT ●●●

40 min

NOTIONS `struct autoréférente` `Noeud **` `parcours`

Une liste chaînée relie des éléments alloués **un par un** : chaque nœud connaît l'adresse du suivant. Plus besoin de `realloc` !

liste.h

```c
typedef struct Noeud {
    int valeur;
    struct Noeud *suivant;
} Noeud;

int  ajouterEnTete(Noeud **tete, int valeur);
void afficherListe(const Noeud *tete);
int  longueur(const Noeud *tete);
void libererListe(Noeud **tete);
```



Après trois ajouts en tête de 10, 20 puis 30.

Ce que tu dois faire

1. `ajouterEnTete` alloue un nœud, le relie à l'ancienne tête, puis met à jour `*tete`.
2. `afficherListe` et `longueur` parcourent la liste avec `p = p->suivant` jusqu'à `NULL`.
3. `libererListe` libère chaque nœud **en sauvegardant le suivant avant** le `free`.
4. Ajoute 10, 20, 30, affiche la liste et sa longueur, puis libère-la.

>_ SORTIE ATTENDUE

```
30 -> 20 -> 10 -> NULL
Longueur : 3
Après libération : liste vide (tete = NULL)
```

Attention

Dans `libererListe`, écrire `free(p); p = p->suivant;` est une erreur classique : on lit un nœud **déjà libéré**. Il faut d'abord mémoriser `p->suivant` dans une variable.

Pour aller plus loin

Ajoute `ajouterEnQueue`, puis `supprimerValeur(Noeud **tete, int v)`. Pourquoi faut-il traiter à part le cas où la valeur est dans le premier nœud ?

NIVEAU 3

30

EXERCICE 30

Projet final : gestionnaire de classe

EXPERT ●●●

60 min

NOTIONS `synthèse` `struct + tableau dynamique` `zéro fuite`

Projet de synthèse du cahier : tout ce que tu as appris se combine. Une classe contient un tableau **dynamique** d'élèves ; chaque élève possède un nom et des notes **dynamiques**.

classe.h

```
typedef struct {
    char   *nom;
    double *notes;
    int     nbNotes;
} Eleve;

typedef struct {
    Eleve *eleves;      // tableau dynamique
    int    nb;
    int    capacite;
} Classe;

int          ajouterEleve(Classe *c, const char *nom, const double *notes, int nbNotes);
double       moyenneEleve(const Eleve *e);
double       moyenneClasse(const Classe *c);
const Eleve *meilleurEleve(const Classe *c);
void         trierParMoyenne(Classe *c);        // ordre décroissant
void         libererClasse(Classe *c);
```

Ce que tu dois faire

1. Initialise la classe vide : `Classe c = {NULL, 0, 0};`.
2. Ajoute les élèves : Alice (14, 16, 12), Bruno (9, 11, 10), Chloé (18, 15, 17), Dylan (12, 13, 8).
3. Affiche chaque moyenne, la moyenne de la classe (moyenne des moyennes) et le meilleur élève.
4. Trie par moyenne décroissante et affiche le classement.
5. Libère **toute** la mémoire et vérifie l'absence de fuite avec `-fsanitize=address` ou Valgrind.

>_ SORTIE ATTENDUE

```
Alice  14.00
Bruno  10.00
Chloé  16.67
Dylan  11.00
Moyenne de la classe : 12.92
Meilleur élève : Chloé (16.67)

Classement : 1. Chloé  2. Alice  3. Dylan  4. Bruno
Mémoire libérée - 0 fuite détectée
```

NIVEAU 3

Conseil

- Découpe : écris et teste `ajouterEleve` + `libererClasse` **en premier**, avec l'outil de détection de fuites. Le reste viendra facilement.
- Pour trier, échange des structures `Eleve` entières : seuls les pointeurs `nom` et `notes` sont copiés, pas les données qu'ils désignent.
- `libererClasse` : pour chaque élève, libère le nom et les notes ; puis libère le tableau ; puis remets la classe à zéro.

Pour aller plus loin

Ajoute un menu interactif (ajouter, afficher, trier, quitter) et la sauvegarde de la classe dans un fichier texte avec `fprintf` .

Auto-évaluation — niveau 3

- [ ] Je sais quand utiliser le tas plutôt que la pile.
- [ ] Je teste chaque `malloc` , `calloc` et `realloc` .
- [ ] J'utilise systématiquement un pointeur temporaire avec `realloc` .
- [ ] Je sais pourquoi une fonction a parfois besoin d'un `Type **` .
- [ ] Je libère les structures imbriquées de l'intérieur vers l'extérieur.
- [ ] Mes programmes passent AddressSanitizer ou Valgrind sans aucune fuite.

ANNEXE A

Aide-mémoire

Pointeurs — l'essentiel

```
int x = 5;
int *p = &x;      // p pointe vers x
*p = 8;           // x vaut 8
int *q = NULL;    // pas de cible
```

- `&x` : adresse · `*p` : valeur pointée
- `%p` + `(void *)` pour afficher

Fonctions et adresses

```
void f(int *s);       // prototype
f(&x);                // appel
*s = *s + 1;          // dans f
```

- Sortie multiple : retour = succès
- Tester `NULL` en entrée

Tableaux

```
tab[i]   ==  *(tab + i)
&tab[i]  ==  tab + i
for (int *p = tab; p < tab + n; p++)
```

- Toujours transmettre la taille
- `const int *` si lecture seule

Structures

```
typedef struct { float prix; } Produit;
Produit a;   a.prix = 2.5f;
Produit *p = &a;   p->prix = 3.0f;
```

- `p->x` ≡ `(*p).x`
- `strcmp` pour comparer des chaînes

Allocation

```
int *t = malloc(n * sizeof *t);
if (t == NULL) { /* erreur */ }
/* ... */
free(t);  t = NULL;
```

- `calloc` : initialise à 0
- 1 malloc = 1 free

realloc et Type **

```
int *tmp = realloc(*t, cap * sizeof *tmp);
if (tmp == NULL) return 0;
*t = tmp;
```

- Jamais `t = realloc(t, …)`
- `Type **` pour modifier un pointeur

Bon travail ! — Institut de Technologie · Ville de Liège · Programmation procédurale 2026 – 2027

ANNEXES

ANNEXE B

Les dix erreurs classiques

Erreur	Symptôme	Correction
Pointeur non initialisé	plantage aléatoire, valeurs absurdes	initialiser avec une adresse ou `NULL`
Déréférencer `NULL`	*segmentation fault*	tester `if (p != NULL)` avant `*p`
Oublier `&` à l'appel	avertissement *incompatible pointer*	`f(&x)` et non `f(x)`
Dépasser un tableau	valeurs corrompues, plantage	boucle `i < taille` et non `i <= taille`
`sizeof` d'un paramètre tableau	taille de 8 au lieu de n	transmettre la taille en paramètre
Comparer des chaînes avec `==`	comparaison toujours fausse	`strcmp(a, b) == 0`
Oublier le `+ 1` du `\0`	chaîne tronquée, débordement	`malloc(strlen(s) + 1)`
`t = realloc(t, …)`	fuite si l'agrandissement échoue	pointeur temporaire
Oublier `free`	fuite mémoire	1 `malloc` = 1 `free`, vérifier avec ASan
Utiliser après `free`	comportement indéfini	`p = NULL;` juste après `free(p)`

C Compiler, déboguer, vérifier

terminal

```
# Compilation recommandée pour tout le cahier
gcc -std=c11 -Wall -Wextra -Wpedantic -g programme.c -o programme

# Avec math.h (exercice 14)
gcc -std=c11 -Wall -Wextra -Wpedantic programme.c -o programme -lm

# Détection des erreurs mémoire (niveau 3)
gcc -std=c11 -Wall -Wextra -g -fsanitize=address,undefined programme.c -o programme
valgrind --leak-check=full ./programme
```

Conseil

Dans CLion : *Settings → Build → CMake*, ajoute `-fsanitize=address` aux options de compilation, ou installe Valgrind et lance *Run with Valgrind Memcheck*.

ANNEXES